

Часть 3. Jobs: параллелизм без race conditions

В прошлой части — анатомия ECS: Entity, Components, Authoring + Baker, ISystem, EntityCommandBuffer.

ECS даёт data-oriented layout памяти, но реальное ускорение приходит когда обработка идёт параллельно на всех CPU-ядрах. За это отвечает **Unity Job System** — отдельный слой, который безопасно распиливает работу на worker-поток.

Разбор паттернов на примере Survival-фичи: top-down arena с волнами врагов и автоматической стрельбой.

Зачем нужны Jobs

Кадр в Unity — это бюджет ~16ms на 60 FPS. Обработка 500 врагов в одном потоке съест львиную долю этого бюджета: на каждого врага — расчёт направления, поворота, позиции, separation от соседей. На main thread это сериальные ~500 итераций.

С Job System та же работа размазывается по N worker-поток (обычно 4-12 в зависимости от CPU). 500 врагов делятся на чанки — каждый поток обрабатывает свою порцию.

Critical: parallel-ускорение **работает только при отсутствии race conditions**. Job System не магически разруливает гонки — он их предотвращает через жёсткую систему safety-проверок.

Виды Jobs

В Unity их несколько, для ECS используются преимущественно два:

Job type	Когда
IJob	Одна задача в фоне (загрузка config'a, фоновая генерация)
IJobFor / IJobParallelFor	Параллельный цикл по индексам в NativeArray
IJobEntity	Основной для ECS — итерация по entity с фильтром по компонентам
IJobChunk	Низкоуровневый chunk-iteration, когда нужен полный контроль

В Survival-фиче 90% job'ов — **IJobEntity**. Source generator превращает структуру с методом `Execute(...)` в полноценный chunk-iterating код, оптимизированный под Burst.

IJobEntity на примере

EnemyMoveJob — каждый кадр считает направление от врага к игроку и сдвигает позицию. Идеальный кандидат на параллелизацию: каждый враг независим от соседа, нет shared state.

```
[BurstCompile]
[WithAll(typeof(EnemyTag))]
```

```

public partial struct EnemyMoveJob : IJobEntity
{
    public float3 PlayerPos;
    public float DeltaTime;

    private void Execute(ref LocalTransform transform,
                        in MoveSpeed speed,
                        in RotationSpeed rotationSpeed,
                        in ContactDamage cd)
    {
        float3 toPlayer = PlayerPos - transform.Position;
        toPlayer.y = 0f;
        float distSq = math.lengthsq(toPlayer);
        if (distSq <= 1e-6f) return;

        float3 direction = math.normalize(toPlayer);

        // Поворот к игроку
        quaternion target = quaternion.LookRotationSafe(direction, math.up());
        transform.Rotation = math.slerp(transform.Rotation, target,
            math.saturate(rotationSpeed.Value * DeltaTime));

        // Движение только если за пределами melee-радиуса
        if (distSq > cd.Radius * cd.Radius)
            transform.Position += direction * speed.Value * DeltaTime;
    }
}

```

Что здесь происходит:

- `[BurstCompile]` — компиляция в нативный SIMD-код (часть 4).
- `[WithAll(typeof(EnemyTag))]` — фильтр: обрабатывать только entities с тегом.
- Параметры `Execute` — компоненты, которые нужны. `ref` = write-access, `in` = read-only.
- Поля job'a (`PlayerPos`, `DeltaTime`) — данные передаваемые **снаружи**, копируются по value в каждый рабочий чанк.

Запуск из системы:

```

[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    new EnemyMoveJob
    {
        PlayerPos = SystemAPI.GetSingleton<PlayerPosition>().Value,
        DeltaTime = SystemAPI.Time.DeltaTime
    }.ScheduleParallel();
}

```

Source Generator увидит `ScheduleParallel()` → автоматически разделит чанки между worker-потоками. Никаких циклов вручную, никаких индексов. Чистый data-oriented код.

Schedule vs ScheduleParallel vs Run

Три способа запустить job:

Метод	Где выполняется	Когда использовать
<code>.Run()</code>	Main thread, синхронно	Только для отладки или когда работа крошечная
<code>.Schedule()</code>	Один worker-поток, асинхронно	Когда есть shared state (RMW) который нельзя параллелить
<code>.ScheduleParallel()</code>	Множество worker-потоков	Когда entities обрабатываются независимо

Главное правило выбора между `.Schedule()` и `.ScheduleParallel()`:

Если несколько entities пишут в один и тот же объект — нельзя параллельно.

Race conditions: ComponentLookup и общий state

`EnemyMoveJob` параллелен потому что **каждый враг пишет только в свой собственный `LocalTransform`**. Никто не трогает соседа.

Контр-пример: несколько зомби в melee одновременно бьют игрока — каждый должен **читать-модифицировать-записать** общий `Health` на entity игрока. Это классический race condition.

Для доступа к компоненту **другой** entity (не той над которой итерирует job) используется `ComponentLookup<T>`:

```
[BurstCompile]
[WithAll(typeof(EnemyTag))]
[WithNone(typeof(DeadTag))]
public partial struct EnemyContactDamageJob : IJobEntity
{
    public Entity PlayerEntity;
    public float3 PlayerPos;
    public float DeltaTime;
    public ComponentLookup<Health> HealthLookup;
    public EntityCommandBuffer Ecb;

    private void Execute(Entity entity, in LocalTransform transform,
        ref ContactDamage cd)
    {
        cd.Timer = math.max(0f, cd.Timer - DeltaTime);

        float2 d = transform.Position.xz - PlayerPos.xz;
        if (math.lengthsq(d) > cd.Radius * cd.Radius) return;
        if (cd.Timer > 0f) return;
```

```

    // Read-modify-write общего Health — НЕ ПАРАЛЛЕЛЬНО
    Health hp = HealthLookup[PlayerEntity];
    hp.Value -= cd.DamagePerHit;
    HealthLookup[PlayerEntity] = hp;

    cd.Timer = cd.Interval;
}
}

```

`ComponentLookup<Health>` — random-access lookup в массив компонентов по entity ID. Запрашивается в системе и обновляется через `Update(ref state)` каждый кадр:

```

public void OnCreate(ref SystemState state)
{
    _healthLookup = state.GetComponentLookup<Health>(false); // false = writable
}

public void OnUpdate(ref SystemState state)
{
    _healthLookup.Update(ref state);

    state.Dependency = new EnemyContactDamageJob
    {
        PlayerEntity = playerEntity,
        PlayerPos = playerPos,
        DeltaTime = SystemAPI.Time.DeltaTime,
        HealthLookup = _healthLookup,
        Ecb = ecb
    }.Schedule(state.Dependency); // ← .Schedule(), не .ScheduleParallel()
}

```

Если попытаться запустить этот job через `.ScheduleParallel()` — Unity Safety System выкинет runtime exception: `ComponentLookup<Health>` с write-access не может работать параллельно. Это hard-error на уровне планирования.

Когда single-thread оказывается быстрее multi-thread

Counter-intuitive вывод: для сценария «много зомби бьют одного игрока» **single-thread .Schedule()** **быстрее** гипотетической параллельной версии.

Причины:

1. **Synchronization overhead.** Чтобы параллельный job безопасно изменял общий Health, потребовалась бы система атомиков или per-frame накопление урона с финальным reduce. Оба варианта добавляют overhead больше чем экономят.
2. **Worker thread cost.** Запуск job'a на worker-поток — это minimum ~50 микросекунд накладных. Для 50 зомби на main thread сама работа занимает 100-200 микросекунд. Делить такую работу

между потоками невыгодно.

3. **Cache locality**. Single thread проходит компоненты линейно, кэш L1 максимально эффективен. Параллельные потоки конкурируют за кэш-линии общего Health.

Правило: **параллельте то, что независимо. Шарите state — оставайтесь на одном потоке.**

NativeContainers и state.Dependency

Job не может держать ссылки на managed-объекты — он работает в Burst-контексте. Для передачи данных между main thread и worker'ами используются **NativeArray**, **NativeList**, **NativeHashMap** — все они аллоцируются в нативной памяти с явным lifetime.

Типичный паттерн: материализовать массив данных перед job'ом, передать в job, диспознуть после завершения:

```
public void OnUpdate(ref SystemState state)
{
    var enemies = _enemyQuery.ToEntityArray(Allocator.TempJob);
    var enemyTransforms = _enemyQuery
        .ToComponentDataArray<LocalTransform>(Allocator.TempJob);

    state.Dependency = new DamageJob
    {
        Enemies = enemies,
        EnemyTransforms = enemyTransforms,
        // ...
    }.Schedule(state.Dependency);

    enemies.Dispose(state.Dependency);
    enemyTransforms.Dispose(state.Dependency);
}
```

Ключевая деталь — **Dispose(state.Dependency)**. Это не моментальный dispose, а **отложенный**: память освободится после того как job завершится. Если сделать **enemies.Dispose()** без зависимости, native-память будет освобождена пока job ещё работает = crash.

state.Dependency — это chain зависимостей всех job'ов системы. Каждый раз когда система планирует job, она цепляет его за **state.Dependency**, и присваивает результат обратно. Unity автоматически выстраивает граф: какие job'ы должны дождаться предыдущих перед стартом.

Типичная ловушка: забыть присвоить результат **Schedule()** обратно в **state.Dependency**. Тогда следующий job не узнает про текущий → race condition или exception от safety-system'ы.

Что дальше

Часть 4 — **Burst**. Что это, как один атрибут превращает managed C# в нативный SIMD-код, и почему без него ECS-производительность снижается в разы.

Связанная литература

- [Unity Job System documentation](#)
- [IJobEntity overview](#)
- Часть 2 серии — «ECS внутри: Components, Systems, Baker»